

GraphQL

1. What is GraphQL?

GraphQL is a query language for APIs that allows clients to request **exactly the data they need**.

Instead of many fixed endpoints, clients query one API endpoint with flexible fields.

Created by Meta.

2. Core Idea

Client decides:

None

What fields are needed

Which nested objects are needed

Response shape

Server returns only requested data.

3. Example Problem in REST

REST call:

None

GET /users/10

Returns:

JSON

```
{  
  "id": 10,  
  "name": "Samarth",  
  "email": "...",  
  "address": "...",  
  "phone": "...",  
  "orders": [...]  
}
```

Client may only need:

None

name

email

Extra data = over-fetching.

4. GraphQL Solution

Query:

None

```
{  
  user(id: 10) {  
    name  
    email  
  }  
}
```

Response:

JSON

```
{  
  "data": {  
    "user": {  
      "name": "Samarth",  
      "email": "sam@example.com"  
    }  
  }  
}
```

Only required data returned.

5. Under-fetching Problem Solved

REST may need:

None

```
GET /user/10
```

```
GET /user/10/orders
```

```
GET /user/10/profile
```

Multiple requests.

GraphQL:

None

```
{  
  user(id:10){  
    name  
    orders { id total }  
    profile { city }  
  }  
}
```

Single request.

6. Main Benefits

None

No over-fetching

No under-fetching

Single endpoint

Flexible frontend development

Strong schema

Self-documenting API

7. One Endpoint Model

Usually:

None

/graphql

All queries/mutations go there.

8. GraphQL Operations

Operation	Purpose
Query	Read data
Mutation	Write/update data
Subscription	Real-time updates

9. Query Example

None

```
query {  
  products {  
    id  
    name  
    price  
  }  
}
```

10. Mutation Example

None

```
mutation {  
  createUser(name:"Samarth"){  
    id  
    name  
  }  
}
```

11. Subscription Example

None

```
subscription {  
  orderStatus(orderId:101){  
    status  
  }  
}
```

Useful for live apps.

12. Schema Driven Design

GraphQL APIs define types.

Example:

```
None
User
Product
Order
Cart
```

Each field has type.

Strong contracts between frontend and backend.

13. Auto Documentation

Most GraphQL servers provide interactive docs/playground.

Examples:

```
None
GraphiQL
Apollo Studio
Playground UI
```


Huge developer advantage.

14. Best Use Cases

Use GraphQL for:

None

Frontend apps needing custom views

Mobile apps with bandwidth limits

Complex relational data

Rapid UI iteration

Multi-device apps

Dashboard systems

15. Example E-commerce Use

Homepage needs:

None

User name

Cart count

Recommended products

Offers

Recent orders

GraphQL fetches all in one query.

16. Why Frontend Teams Like It

None

Frontend controls response shape

Backend changes less frequently

Less waiting for new endpoints

Faster feature development

17. Tradeoffs

None

More complex backend

Harder caching than REST

Query cost management needed

N+1 DB query risk

Security needs limits

Learning curve higher

18. Security Concerns

Need controls for:

None

Deep nested queries

Huge expensive joins

Too many fields requested

Abuse by public clients

Rate limiting

Auth per field

19. Analytics Harder

REST:

None

GET /products = easy count

GET /orders = easy count

GraphQL:

None

Every client sends different query

Harder to measure usage patterns.

20. GraphQL vs REST vs RPC

Feature	GraphQL	REST	RPC
Flexible fields	High	Low	Low
Human readable	High	High	Medium
Speed	Medium	Medium	High
Strong schema	High	Medium	High
Easy caching	Medium/Low	High	Medium

21. When Not Ideal

Avoid GraphQL for:

None

Very simple CRUD APIs

Tiny internal services

High-performance binary communication

Simple public APIs

REST/RPC may be better.

22. Internal Architecture

GraphQL often acts as API gateway:

None

Frontend



GraphQL Gateway



User Service

Order Service

Payment Service

Inventory Service

23. Example Query for Dashboard

None

```
{  
  user(id:5){  
    name  
    orders { total }  
    wishlist { name price }  
  }  
}
```

Single round-trip.

24. Interview Talking Points

Say GraphQL when system needs:

None

Multiple clients with different data needs

Complex UI screens

Reduce API calls

Frontend flexibility

Nested relational data

25. One-Line Summary

GraphQL is an API query language where clients request exactly the fields they need, reducing over-fetching and enabling flexible, schema-driven frontend development.